🎯 10 ESSENTIAL OOP QUESTIONS (Q8-Q17)

Post Q1-Q7 Mastery Questions

Covers: Encapsulation, Static/Final, Overloading, Inheritance, Polymorphism, Abstraction, Interfaces, Real-world Design

---

Q8: ENCAPSULATION WITH VALIDATION + GETTERS/SETTERS

[Covers: Private fields, Getters, Setters, Validation, immutability]

Create a `Student` class:

Requirements:

- Properties: `name` (String), `gpa` (float 0-4.0), `age` (int 18-60)
- Private fields (all properties should be PRIVATE)
- Parameterized constructor with validation
- Getter methods (read-only access)
- Setter methods with validation
  - `setGpa()` - Should only accept 0-4.0, reject others
  - `setAge()` - Should only accept 18-60, reject others
- A `final` String field `universityName` that can't be changed
- `displayInfo()` method

Test in main():

```java

```

```
Student s1 = new Student("Yash", 3.8f, 20);
s1.displayInfo();

// Test getters
System.out.println("GPA: " + s1.getGpa());

// Test setter with valid value
s1.setGpa(3.9f);
s1.displayInfo();

// Test setter with invalid value (should reject)
s1.setGpa(5.0f);  // Should print error
s1.displayInfo();  // GPA should remain 3.9

// Test final field (should give compile error)
// s1.universityName = "NewUniversity";  // ERROR!
```

Key Concepts Tested:

- ✅ Encapsulation (private fields)
- ✅ Getters (read-only)
- ✅ Setters with validation
- ✅ Final fields
- ✅ Immutability

Q9: STATIC & FINAL KEYWORDS (Combined)

[Covers: Static variables, Static methods, Final variables, Final methods, Static blocks]

Create a `Database` class:

Requirements:

- Static variable `connectionCount` (tracks total connections)
- Static variable `MAX_CONNECTIONS = 10` (constant, never changes)
- Instance variable `databaseName` (each database has its own)
- Static block that initializes default settings
- Constructor that increments `connectionCount`
- Static method `displayTotalConnections()` to show total
- Instance method `connect()` that uses static MAX_CONNECTIONS
- Final method `close()` that can't be overridden

Test in main():

```java
System.out.println("Max connections allowed: " + Database.MAX_CONNECTIONS);

Database db1 = new Database("MySQL");
Database.displayTotalConnections();  // Should show 1

Database db2 = new Database("PostgreSQL");
Database.displayTotalConnections();  // Should show 2

// Try to change MAX_CONNECTIONS (should error)
// Database.MAX_CONNECTIONS = 20;  // ERROR!
```

Key Concepts Tested:

- ✅ Static variables (shared by all objects)
- ✅ Static methods (call without object)
- ✅ Static block (initialization)
- ✅ Final constants
- ✅ Final methods (can't override)
- ✅ Static vs Instance context

Q10: METHOD OVERLOADING (All Types Combined)

[Covers: Parameter type overloading, Parameter count, Varargs, Return type doesn't matter]

Create a `Calculator` class with overloaded `calculate()` methods:

Requirements:

- `calculate(int a, int b)` → Addition
- `calculate(int a, int b, int c)` → Addition of 3 numbers
- `calculate(double a, double b)` → Multiplication
- `calculate(String op, int a, int b)` → Operation based on String ("+", "-", "*", "/")
- `calculate(int... numbers)` → Sum of variable arguments
- Constructor overloading: `Calculator()`, `Calculator(String name)`

Test in main():

```java
```

```
Calculator c1 = new Calculator();
Calculator c2 = new Calculator("MyCalculator");

System.out.println(c1.calculate(5, 3));              // int: 8
System.out.println(c1.calculate(5, 3, 2));           // int: 10
System.out.println(c1.calculate(5.0, 3.0));          // double: 15.0
System.out.println(c1.calculate("+", 5, 3));         // String: 8
System.out.println(c1.calculate(1, 2, 3, 4, 5));     // varargs: 15
```

Key Concepts Tested:

- ✅ Method overloading by type
- ✅ Method overloading by count
- ✅ Method overloading by parameter order
- ✅ Varargs (variable arguments)
- ✅ Constructor overloading

Q11: INHERITANCE (Single + Super Keyword)

[Covers: extends, super(), super method call, constructor chaining in inheritance]

Create a inheritance hierarchy:

Requirements:

```
Parent Class: Vehicle
├─ Properties: name, color, year
├─ Constructor: Vehicle(String name, String color, int year)
├─ Method: displayInfo()
└─ Method: start() → "Vehicle started"

Child Class: Car
├─ Additional Property: numberOfDoors
├─ Constructor: Car(String name, String color, int year, int numberOfDoors)
│  └─ Use super() to initialize parent
├─ Override: displayInfo() - call super.displayInfo() first, then add car-
specific info
├─ Override: start() - call super.start(), then add "Car is starting"
└─ Additional Method: honk() → "Car honking"
```

Test in main():

java

```
Vehicle v1 = new Vehicle("GenericVehicle", "Red", 2020);
v1.displayInfo();
v1.start();

Car c1 = new Car("Honda Civic", "Blue", 2021, 4);
c1.displayInfo();   // Should show parent info + doors
c1.start();         // Should call parent's start + car-specific
c1.honk();
```

Key Concepts Tested:

- ✅ Single inheritance (extends)
- ✅ super() in constructor
- ✅ super.method() call
- ✅ Method overriding
- ✅ Parent and child properties

Q12: METHOD OVERRIDING + POLYMORPHISM (Runtime)

[Covers: Method overriding, Upcasting, Downcasting, instanceof, Late binding]

Create a payment processing system:

Requirements:

```
Parent Class: Payment
├─ Property: amount
├─ Constructor: Payment(float amount)
├─ Abstract idea: process()
└─ Method: displayAmount() → prints amount

Child Classes (implement differently):
├─ CreditCardPayment extends Payment
│   └─ process() → "Processing via Credit Card"
├─ DebitCardPayment extends Payment
│   └─ process() → "Processing via Debit Card"
└─ UPIPayment extends Payment
    └─ process() → "Processing via UPI"
```

Test in main():

java

```
// Array of different payment types (Polymorphism)
Payment[] payments = new Payment[3];
payments[0] = new CreditCardPayment(1000);
payments[1] = new DebitCardPayment(2000);
payments[2] = new UPIPayment(3000);

// Process all using same method call (Late binding)
for (Payment p : payments) {
    p.displayAmount();
    p.process();  // Different implementation for each!
    System.out.println();
}

// Downcasting example
Payment p = new CreditCardPayment(5000);
if (p instanceof CreditCardPayment) {
    CreditCardPayment cp = (CreditCardPayment) p;
    // Use CreditCardPayment specific methods
}
```

Key Concepts Tested:

- ✅ Method overriding
- ✅ Polymorphism (runtime)
- ✅ Upcasting (implicit)
- ✅ Downcasting (explicit with instanceof)
- ✅ Late binding
- ✅ Collections with different types

Q13: ABSTRACT CLASSES (Complete Concept)

[Covers: Abstract class, Abstract methods, Concrete methods, Constructor, Template method]

Create an abstract shape system:

Requirements:

```
Abstract Class: Shape (Can't instantiate)
├─ Abstract Property: Abstract method area()
├─ Abstract Property: Abstract method perimeter()
├─ Concrete Method: printShape() → "This is a shape"
├─ Constructor: Shape(String name)
│  └─ Initialize name
└─ Concrete Method: displayAll() (Template Method)
   ├─ printShape()
   ├─ Call area()
   └─ Call perimeter()

Concrete Class: Circle extends Shape
├─ Property: radius
├─ Constructor: Circle(String name, float radius)
│  └─ Use super()
├─ Override: area() → πr²
└─ Override: perimeter() → 2πr

Concrete Class: Rectangle extends Shape
├─ Properties: length, width
├─ Override: area() → l*w
└─ Override: perimeter() → 2(l+w)
```

Test in main():

java

```java
// Can't do: Shape s = new Shape();  // ERROR!

Shape c = new Circle("MyCircle", 5);
c.displayAll();

Shape r = new Rectangle("MyRectangle", 4, 6);
r.displayAll();

// Array of shapes
Shape[] shapes = {new Circle("C1", 3), new Rectangle("R1", 4, 5)};
for (Shape s : shapes) {
    s.displayAll();
}
```

Key Concepts Tested:

- ✅ Abstract class (can't instantiate)
- ✅ Abstract methods (must override)
- ✅ Concrete methods in abstract class

- ✅ Constructor in abstract class
- ✅ Template method pattern
- ✅ Polymorphism with abstract

Q14: INTERFACES (Complete Concept)

[Covers: Interface contract, Multiple interfaces, Default methods, Static methods]

Create a device system:

Requirements:

```
Interface 1: Chargeable
├─ Abstract Method: charge()
├─ Abstract Method: getChargePercentage()
└─ Default Method: displayChargeStatus()

Interface 2: Connectable
├─ Abstract Method: connect(String deviceName)
└─ Abstract Method: disconnect()

Concrete Class: Smartphone implements Chargeable, Connectable
├─ Properties: deviceName, chargePercentage
├─ Constructor: Smartphone(String deviceName)
├─ Implement charge(): Increase chargePercentage
├─ Implement getChargePercentage(): Return current percentage
├─ Implement connect(): "Connected to " + deviceName
└─ Implement disconnect(): "Disconnected from device"

Concrete Class: Laptop implements Chargeable, Connectable
├─ Similar implementations as Smartphone
```

Test in main():

```java

```

```
Smartphone phone = new Smartphone("iPhone");
phone.charge();
phone.displayChargeStatus();  // From default method
phone.connect("WiFi");
phone.disconnect();

Laptop laptop = new Laptop("Dell");
laptop.charge();
laptop.displayChargeStatus();
laptop.connect("Ethernet");

// Interface type reference
Chargeable[] devices = {phone, laptop};
for (Chargeable device : devices) {
    device.charge();
    device.displayChargeStatus();
}
```

Key Concepts Tested:

- ✅ Interface contract
- ✅ Multiple interface implementation
- ✅ Abstract methods in interface
- ✅ Default methods (Java 8+)
- ✅ Polymorphism with interfaces
- ✅ Vs abstract class

Q15: COMPLEX POLYMORPHISM + INHERITANCE HIERARCHY

[Covers: Multi-level inheritance, Method overriding chain, Polymorphic behavior]

Create an employee hierarchy:

Requirements:

```
Level 1: Employee (Abstract)
├─ Properties: name, salary
├─ Abstract: calculateBonus()
├─ Concrete: displayInfo()
└─ Constructor: Employee(String name, float salary)

Level 2: Manager extends Employee (Abstract)
├─ Property: teamSize
├─ Override: calculateBonus() → salary * 0.15 * (teamSize/10)
└─ Constructor: Manager(String name, float salary, int teamSize)

Level 3: ProjectManager extends Manager
├─ Property: projectsHandled
├─ Override: calculateBonus() → Super + (projectsHandled * 1000)
└─ Constructor: ProjectManager(String name, float salary, int teamSize, int
projectsHandled)

Level 3: HRManager extends Manager
├─ Override: calculateBonus() → salary * 0.2
└─ No additional properties
```

Test in main():

java

```java
// Can't instantiate: Employee e = new Employee();

Employee[] employees = {
    new ProjectManager("Yash", 50000, 5, 3),
    new HRManager("Aniket", 45000, 10),
    new ProjectManager("Priya", 60000, 8, 5)
};

// Polymorphic call - different calculation for each!
for (Employee emp : employees) {
    emp.displayInfo();
    System.out.println("Bonus: " + emp.calculateBonus());
}
```

Key Concepts Tested:

- ✅ Multi-level inheritance
- ✅ Abstract classes in hierarchy
- ✅ Method overriding chain
- ✅ Polymorphism in action

- ✅ Super keyword in chain

Q16: EQUALS() & HASHCODE() (Collections & Comparison)

[Covers: Object comparison, equals override, hashCode override, Collections]

Create a `Book` class:

Requirements:

- Properties: `title`, `author`, `isbn`
- Constructor: `Book(String title, String author, String isbn)`
- Override `equals()` → Two books are equal if they have the same ISBN
- Override `hashCode()` → Based on ISBN
- Override `toString()` → Display book info
- Regular getters

Test in main():

```java
Book b1 = new Book("Chhava", "Shivaji Sawant", "ISBN-001");
Book b2 = new Book("Chhava", "Shivaji Sawant", "ISBN-001");  // Different objec
Book b3 = new Book("Shrimanyogi", "Shivaji Sawant", "ISBN-002");

// Test equals
System.out.println(b1.equals(b2));   // true (same ISBN)
System.out.println(b1.equals(b3));   // false (different ISBN)

// Test hashCode
System.out.println(b1.hashCode() == b2.hashCode());  // true (same ISBN)

// Use in HashSet (requires equals + hashCode)
HashSet<Book> library = new HashSet<>();
library.add(b1);
library.add(b2);  // Should not be added (equals returns true)
library.add(b3);

System.out.println("Total unique books: " + library.size());  // Should be 2
```

Key Concepts Tested:

- ✅ equals() override
- ✅ hashCode() override
- ✅ Relationship between equals and hashCode

- ✅ Collections (HashSet)
- ✅ Object comparison

Q17: REAL-WORLD DESIGN PROBLEM (Integration of All Concepts)

[Covers: Everything - Encapsulation, Inheritance, Polymorphism, Abstraction, Interfaces, Static, Final]

Choose ONE and build complete system:

Option A: Library Management System

```
Abstract Class: LibraryItem
├─ Properties: id (static final auto-increment), title, author, year,
isAvailable
├─ Constructor: LibraryItem(String title, String author, int year)
├─ Static: totalItems, totalIssued
├─ Abstract: getDueDate()
├─ Concrete: issueItem(), returnItem()

Interface: Searchable
├─ searchByTitle(String title)
└─ searchByAuthor(String author)

Interface: Borrowable
├─ borrow(String memberName)
└─ return()

Classes: Book, Magazine, Newspaper (extend LibraryItem, implement Searchable,
Borrowable)

Class: Library
├─ Properties: List<LibraryItem> items, List<String> members
├─ Static method: getTotalLibraryStats()
└─ Methods: addItem(), issueItem(), searchItem()
```

Main requirements:

- Private fields with proper getters/setters
- Validation (dates, availability, member existence)
- Polymorphism (process different item types same way)
- Static tracking (total items, total issued)
- Collections (ArrayList/HashSet)

Option B: E-Commerce Cart System

```
Interface: Discountable
├─ getDiscount()
└─ applyDiscount()

Abstract Class: Product
├─ Properties: id, name, price (private final), category
├─ Abstract: getDescription()
├─ Static: totalProducts
├─ Constructor: Product(String name, float price, String category)

Classes: Electronics, Clothing, Groceries (extend Product)

Interface: Orderable
├─ addToCart()
└─ removeFromCart()

Class: ShoppingCart
├─ Properties: List<Product>, totalPrice
├─ Methods: addItem(), removeItem(), calculateTotal() with discounts
├─ Collections: HashMap for product quantities

Class: Order
├─ Static: orderId (auto-increment)
├─ Final: orderDate
├─ Method: generateReceipt()
```

Test Requirements:

java

```
// Create different products
Product p1 = new Electronics("Laptop", 50000, "Computers");
Product p2 = new Clothing("T-Shirt", 500, "Apparel");
Product p3 = new Groceries("Rice", 150, "Food");

// Add to cart
ShoppingCart cart = new ShoppingCart();
cart.addItem(p1, 1);
cart.addItem(p2, 2);
cart.addItem(p3, 3);

// Calculate total with polymorphic discount
System.out.println("Total: " + cart.calculateTotal());

// Create order
Order order = new Order(cart);
order.generateReceipt();

// Search/Filter
cart.searchByCategory("Computers");
```

Key Concepts Tested:

- ✅ Encapsulation (private fields, validation)
- ✅ Inheritance (hierarchy of products/items)
- ✅ Polymorphism (same method, different behavior)
- ✅ Interfaces (contracts for behaviors)
- ✅ Abstract classes (template)
- ✅ Static (counters, auto-increment)
- ✅ Final (immutable fields)
- ✅ Collections (List, HashMap)
- ✅ Complex business logic

📋 SUMMARY OF Q8-Q17 COVERAGE

Question	Topic	Key Focus
Q8	Encapsulation	Getters, Setters, Validation, Final
Q9	Static & Final	Static vars, methods, blocks, constants
Q10	Method Overloading	Types, count, varargs, constructors

Question	Topic	Key Focus
Q11	Inheritance	extends, super(), method override
Q12	Polymorphism	Runtime polymorphism, upcasting, downcasting
Q13	Abstract Classes	Abstract methods, template method pattern
Q14	Interfaces	Multiple interfaces, default methods
Q15	Complex Hierarchy	Multi-level inheritance, chains
Q16	Collections	equals(), hashCode(), HashSet
Q17	Real-World Design	Integration of ALL concepts

🎯 How to Approach

1. **Q8-Q9:** Fundamental OOP principles (data protection)
2. **Q10:** Method design flexibility
3. **Q11-Q12:** Code reuse and polymorphism
4. **Q13-Q14:** Abstraction and contracts
5. **Q15:** Complex relationships
6. **Q16:** Practical collections usage
7. **Q17:** Placement-level design problem

🚀 Interview Preparation Note

These 10 questions cover:

- ✅ 90% of OOP interview questions
- ✅ All SOLID principles concepts
- ✅ Real-world application scenarios
- ✅ Data structure + OOP combined

After these, you'll be **interview-ready for any Java OOP question!**

Start with Q8! Let's build your OOP mastery step by step. 💪